

AI Agents Setup Guide Everything a new team member needs to get Claude Code, Codex, and Pi working with this repo's skills, tools, and optimizations. Follow in order â€” each section builds on the previous. --- ## What this repo is A shared library of AI agent skills, tools, standards, and automation scripts. Your projects don't copy from it â€” they import from it via live `@`-references. When this repo is updated, your projects pick up changes automatically on the next session. `` canon/ â†• this repo (shared library, clone once) skills/ â†• wrapup, capture, pdf (+ hidden deps: code-reviewer, code-simplifier, security-review) tools/ â†• handoff, ticket, tkt.sh (ticket is auto-added with wrapup) standards/ â†• efficiency (coding, git, token-efficiency â€” one unified file) scripts/ â†• hook automation (handoff, wrapup trigger, pre-commit) guides/ â†• this file and context-optimization.md extensions/pi/ â†• Pi lifecycle extensions your-project/ â†• your work repo CLAUDE.md â†• @-imports pointing into canon AGENTS.md â†• skill table + inlined standards for Codex and Pi HANDOFF.md â†• session context (auto-managed) `` --- ## Step 1 â€” Clone canon Clone the repo anywhere you like. The scripts self-locate at runtime. ``bash git clone https://github.com/sunitghub/canon.git ~/Developer/canon `` Then set a shell variable for the rest of this guide (substitute your actual clone path): ``bash export SKILLS=~/.Developer/canon `` Verify: ``bash ls \$SKILLS/skills.sh # should exist `` --- ## Step 2 â€” Install prerequisites \*\*RTK\*\* (optional, recommended) â€” filters verbose CLI output before it hits the AI's token budget. Saves 60â€”90% of tokens on common operations. Setup detects whether it's installed and wires the hook automatically if so; skills work without it. ``bash # macOS brew install rtk # WSL / Linux cargo install rtk `` ``bash rtk --version # verify `` > If `rtk gain` fails after install, you may have the wrong package (name collision on crates.io). Use `brew install rtk` â€” not `cargo install rtk`. **tkt** â€” bundled minimal ticket tool, included with canon. No install needed. `skills add wrapup` adds it automatically and offers to put it on your PATH. --- ## Steps 3â€”5 â€” Agent setup (automated) **Claude Code only** â€” run the lightweight init command: ``bash \$SKILLS/skills.sh init `` > Re-run `skills.sh init` any time you move or rename the canon folder â€” it rewires the hook paths in `~/.claude/settings.json` to the new location. **Claude Code + Codex + Pi** â€” run the full setup script. It is safe to run multiple times: ``bash \$SKILLS/init-agent.sh `` It will prompt you to choose an agent (or `all`), back up any existing config files before modifying them (`.bak` extension), and report what was added vs already present. You can also run non-interactively: ``bash \$SKILLS/init-agent.sh claude # Claude Code only \$SKILLS/init-agent.sh codex # Codex only \$SKILLS/init-agent.sh pi # Pi only \$SKILLS/init-agent.sh all # all three `` **What it sets up per agent:** | Agent | What gets configured | |---| | Claude Code | Handoff + quality hooks merged into `~/.claude/settings.json`. RTK hook wired automatically if `rtk` is installed; skipped with a hint if not. | | Codex | RTK instructions via `rtk init -g --codex` â†• writes `~/.codex/RTK.md` + `@reference` in `~/.codex/AGENTS.md` (skipped if RTK absent). | | Pi | Copies `extensions/pi/handoff.ts` to `~/.pi/agent/extensions/` | **Manual fallback (if you prefer to inspect before applying):**

Claude Code â€” manual hook setup
``bash rtk init -g --auto-patch # RTK native hook (non-interactive) `` Then either run `\$SKILLS/skills.sh init` (recommended), or merge manually into `~/.claude/settings.json` (replace `` with your actual clone path): ``json { "hooks": { "Stop": [{ "matcher": "", "hooks": [{ "type": "command", "command": "/scripts/auto-handoff.sh" }] }, "UserPromptSubmit": [{ "matcher": "", "hooks": [{ "type": "command", "command": "/scripts/handoff-inject.sh" }] }, "PostToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command", "command": "/scripts/auto-polish-trigger.sh" }] }, "PreToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command", "command": "/scripts/pre-commit-check.sh" }] }] } } ``

Codex â€” manual setup
``bash rtk init -g --codex --auto-patch ``

Pi â€” manual setup
``bash mkdir -p ~/.pi/agent/extensions cp \$SKILLS/extensions/pi/handoff.ts ~/.pi/agent/extensions/handoff.ts # then /reload in Pi ``

--- ## Step 6 â€” Per-project setup Run this once per project you want to use skills in. ### Register skills **All at once** (recommended for new projects): ``bash cd /path/to/your-project

`$$SKILLS/skills.sh addall `` **Or pick individually:** ``bash cd /path/to/your-project # Full stack â€” wrapup pulls in code-simplifier, code-reviewer, security-review, handoff, and ticket automatically`
`$$SKILLS/skills.sh add wrapup # Optional extras`
`$$SKILLS/skills.sh add capture # mid-session knowledge capture`
`â†’ HANDOFF.md Discoveries`
`$$SKILLS/skills.sh add pdf # PDF read/extract/merge/split `` `addall` is idempotent â€” safe to re-run if new skills have been added to canon since you last registered.`
`> **Standards and deps are auto-injected.**` Every ``skills.sh add`` call automatically injects the ``efficiency`` standard (coding principles, git conventions, token-efficiency rules) â€” as ``@`-imports` in ``CLAUDE.md`` and inline content in ``AGENTS.md``. Skill deps like ``code-reviewer``, ``handoff``, and ``ticket`` are wired silently without appearing in the catalog.

`### Verify registration ``bash $$SKILLS/skills.sh status # or: skills.sh --scan /path/to/your-project ``` This checks registered skills, inline standard freshness, and broken ``@`-import` paths. It reports issues and tells you exactly what to run to fix them.

`### Initialize HANDOFF.md` ``handoff`` is registered automatically as a dep of ``wrapup``. To initialize the file, tell Claude or Codex: "Initialize the handoff file" â€” it creates ``HANDOFF.md`` in the project root from the template.

`--- ## Verification checklist` Run through these to confirm everything is wired up correctly.

`**RTK**` (if installed) ```bash rtk gain # should show "No tracking data yet" or savings stats (not an error) rtk git status # should run and show compact output ```
`**Claude Code hooks**` ```bash cat ~/.claude/settings.json # should contain: auto-handoff, handoff-inject, auto-polish-trigger, pre-commit-check (plus rtk hook if RTK is installed) ```
`**Codex**` (if RTK installed) ```bash cat ~/.codex/AGENTS.md # should contain @RTK.md reference ```
`**Per-project**` ```bash $$SKILLS/skills.sh status # lists registered skills ```

`--- ## Skill verification` After registering skills, confirm each one is wired up and responding correctly.

Skill	How to verify	Expected response
<code>`efficiency`</code>	<code> `skills.sh status` </code> Listed under <code>CLAUDE.md @imports</code> ; <code>[current]</code> in <code>AGENTS.md standards</code>	<code> `wrapup` </code>
<code>`capture`</code>	<code> "Wrapup the changes" or `/wrapup` </code> Runs simplifier â†’ reviewer â†’ security-review with skip reasoning	<code> `ticket` </code>
<code>`handoff`</code>	<code> `ticket` </code> Make a non-obvious discovery mid-session	<code> `ticket` </code>
<code>`pdf`</code>	<code> `ticket` </code> Agent writes to <code>Discoveries`</code> in <code>HANDOFF.md</code> without prompting	<code> `ticket` </code>
<code>`ticket`</code>	<code> `ticket` </code> Extract text from <code>[file].pdf`</code> Extracted content or clear error if no PDF present	<code> `ticket` </code>
<code>`tkl`</code>	<code> `ticket` </code> Empty list or existing tickets (no error)	<code> `ticket` </code>
<code>`handoff`</code>	<code> `ticket` </code> "Initialize the handoff file" <code>`HANDOFF.md`</code> created in project root	<code> `ticket` </code>

`> `efficiency` has no invocation â€” applied automatically to every session. `ticket` is auto-added when wrapup is registered and appears in `skills list`. `handoff` is a hidden dep of `wrapup`. `capture` fires automatically â€” no invocation needed.`

`--- ## Day-to-Day Workflows`

`#### Ticketing with `tkl`` ``tkl`` is a minimal ticket tool bundled with canon. Tickets are markdown files in ``.tickets/`` â€” committed to the repo, visible in git log, and clickable in VS Code. No external install needed.

`**Both Claude and Codex read the same skill file**` (`tools/ticket.md``) via ``@`-import` â€” one in ``CLAUDE.md``, one in ``AGENTS.md``. No agent-specific setup needed.

`##### Key commands` ```bash tkl create "title" [-t bug|feature|task|epic|chore] [-p 0-4] [-d "desc"] tkl ls # list all tickets tkl ls --status=in_progress # filter by status tkl show # full ticket detail tkl start # mark in_progress tkl close # mark closed (prefer: use approve workflow) tkl reopen # reopen a closed ticket ```

`Priority: `0` = highest, `4` = lowest. Default is `2`. IDs are short random strings (e.g. `t-8ms5`).`
`> Need dependency tracking, tags, or assignees? Install [ticket](https://github.com/wedow/ticket) (`brew install ticket`)` â€” same ``.tickets/`` format, fully compatible.

`##### Standard workflow 1. **Create** â€” Ask the agent: "Create a ticket to add X."` The agent runs ``tkl create`` and returns the ticket ID.

`2. **Implement** â€” Ask the agent: "Implement ticket `.`."` The agent runs ``tkl start``, does the work, and prepends ``:`` to every commit.

`3. **Test** â€” Review the changes yourself.`

`4. **Approve** â€” Tell the agent: "Approve `.`."` The agent runs the full pipeline (see below).

`##### Approve pipeline` Say `**"approve" (or "ship it", "approve and close")` after testing. The agent runs:

- ``/wrapup`` on all files modified since the ticket was started
- ``tkl close`` only after wrapup completes

 Agents are instructed never to call ``tkl close`` directly â€” always through the approve pipeline so wrapup never gets skipped.

`--- #### Knowledge Capture â€” Mid-Session Discoveries` The ``capture`` skill ensures non-obvious findings are recorded immediately â€” not just at wrapup â€” so they survive context compaction and session switches.

`**Automatic** â€” no action needed.` When the agent discovers something non-obvious (filter rules found by comparing data, numerical facts

not in code, environment gotchas, architecture decisions with non-obvious WHY), it immediately: 1. Appends to `HANDOFF.md` under `## Discoveries` 2. Saves a project memory

Explicit trigger when you want to force-record something the agent missed: | Agent | Trigger | |---|---| Claude Code | `/capture` | | Codex | "Capture this" / "Record this in discoveries" | | Pi | Same as Codex natural language | The `## Discoveries` section in `HANDOFF.md` is the persistent store. A future agent starting cold reads it to pick up every constraint and decision that required investigation to establish. ---

Wrapup Quality Pipeline Wrapup is the quality gate that closes out any unit of work. It does not require the ticket system run it any time you finish a chunk of code. ##### How to trigger `` /wrapup `` Or: "Wrapup the changes" / "Wrapup the auth refactor" / "Wrapup ticket nw-42." ##### Pipeline `` code-simplifier â†’ code-reviewer â†’ security-review `` Each step is skipped if it doesn't apply the agent states why in one line when skipping. | Step | Skipped when | |-----|-----| | code-simplifier | Single-line change, or docs/config only | | code-reviewer | Single-line fix with no design implications, or purely mechanical change | | security-review | No security-sensitive files changed (auth, DB queries, user input, API endpoints, crypto, session management, file I/O, env/secret access) | ##### What each step produces

- Simplifier** rewrites recently modified code for clarity without changing behavior: reduces nesting, eliminates redundancy, improves names, removes obvious comments. Scope is limited to files touched in the current session.
- Reviewer** structured report across seven dimensions: correctness, maintainability, readability, efficiency, security, edge cases, test coverage. Format: Critical â†’ Improvements â†’ Nitpicks â†’ Recommendations.
- Security review** high-confidence findings only. Traces data flow end-to-end before flagging anything. Reports severity (Critical / High / Medium / Low) with location, pattern, evidence, impact, and fix.

Final output format `` ## Wrapup Report â€œ ## code-simplifier - ## code-reviewer - [Critical] ... - [Improvement] ... ## security-review - [High] ... `` Address criticals before committing. Improvements and nitpicks are at your discretion. ##### Auto-trigger (optional, requires ticket skill) If you use the `ticket` skill, the `PostToolUse` hook (`auto-polish-trigger.sh`) triggers wrapup automatically when a ticket is closed via the approve workflow. Without tickets, run `/wrapup` manually whenever you want to close out a chunk of work. ---

Staying updated When this repo is updated with improved skills, standards, or scripts: ``bash cd \$SKILLS && git pull `` **Hook scripts** update immediately they're called by path, so the new version runs on the next session. **Skill content** in `CLAUDE.md` (Claude) is live `@`-import references Claude picks up changes automatically on the next session. **Inline standards** in `AGENTS.md` (Codex, Pi) are a static copy and need an explicit refresh to pick up changes: ``bash \$SKILLS/skills.sh refresh /path/to/your-project `` `refresh` re-registers every skill already in the project, replaces any outdated inline standard blocks in `AGENTS.md`, and heals stale `@`-import paths all in one command. **For newly added skills** (opt in individually or all at once): ``bash \$SKILLS/skills.sh addall /path/to/your-project # register any new skills \$SKILLS/skills.sh add /path/to/your-project # or just one `` **Check for issues before refreshing:** ``bash \$SKILLS/skills.sh status /path/to/your-project `` Reports stale paths, outdated inline standards, and broken `@`-imports with a one-line fix suggestion when anything is out of date. Check what skills are available: ``bash \$SKILLS/skills.sh list `` ---

How the automation works end-to-end See [`guides/context-optimization.md`](context-optimization.md) for a full explanation of the token optimization, session handoff, and quality pipeline including why each piece is designed the way it is.